Memory Efficient Transformer Attention on CPUs Using Tiling and Online Softmax

CEG5206 Team Project Proposal | 30 September 2026

Group number: 11

Team: 4 students Duration: 6 weeks Platform: CPU

Focus: Cache tiling, online softmax, memory efficiency, SIMD, multicore parallelism

1 Project overview

We propose a controlled performance study of dense Transformer attention [1]. We will implement a materialized baseline, a cache-blocked version, and a streaming version using online softmax. The study will explain how these choices affect execution time, temporary storage, numerical accuracy, and multicore scaling.

Driving question: When do cache tiling and avoiding storage of the full attention matrix improve CPU performance? How do sequence length, feature dimension, tile size, and thread count affect these gains?

Online normalization [2], memory-efficient attention [3], and FlashAttention [4] guide the streaming design. We will implement and analyze these established ideas on CPUs, with reproducible results. We will evaluate CPU performance directly, as GPU speedups from prior work may not transfer to CPUs.

Connection to CEG5206

L01 provides methods for analyzing work and space. L02 and L03 cover memory hierarchy, locality, cache tiling, and working sets. L04 provides work/span and parallelism analysis. L06 covers maximum and sum reductions used in softmax. We will account for the non-associativity of floating-point addition. We will also explain dependencies within each query block and parallelism across query blocks.

2 Scope and computation contract

$$O = softmax(\frac{QK^T}{\sqrt{d}})V$$

Q, K, and V are supplied dense matrices with N rows and d columns. The output O has the same shape. Softmax operates across each row of the N by N score matrix. All implementations compute the same dense, unmasked attention function. Floating-point rounding differences are allowed.

We will use C++17 with FP32 inputs and accumulation. Matrices will use row-major storage with contiguous features. We will use batch size one and one attention head. OpenMP will provide CPU threading. These are proposed engineering boundaries. Before experiments, we will record the CPU, instruction set (ISA), compiler, and dependency versions.

The core excludes Q/K/V projection layers, positional encoding, and full Transformer models. It also excludes training, backward propagation, GPU kernels, masking, dropout, sparse attention, and approximate attention. The team will implement the project kernels. External numerical libraries will support independent validation or clearly labelled reference comparisons.

3 Required implementation tasks

Task 1 Materialized baseline and reference

Implement a straightforward single-thread kernel for QK^T, stable row-wise softmax, and multiplication by V. Subtract the row maximum before exponentiation. Store probabilities in the score buffer, so the main temporary matrix occupies $4N^2$ bytes. Build an independent FP64 NumPy reference using the exact FP32 input values.

Task 2 Cache blocking and vectorization

Implement our own blocked matrix kernels while retaining the full score matrix. Reorder loops and pack panels when data reuse justifies the cost [6]. Use compiler vectorization or intrinsics in suitable feature loops. Check vectorization reports or generated code, and document how remaining elements are handled. Use the same optimization and ISA settings for the baseline and blocked versions. Keep compiler optimization enabled for the baseline.

Task 3 Streaming attention with online softmax

Process Br query rows against successive blocks of Bc keys and values. Retain only a score tile and running statistics for each query. For one query row, m is the running maximum and ℓ is the normalized exponential sum. The vector u stores the unnormalized weighted sum of values. For the next block of scores s, update [2–4]:

$$m' = max(m, max(s)), \quad \alpha = exp(m - m'), \quad p = exp(s - m')$$

$$l' = \alpha l + \sum_j p_j, \quad u' = \alpha u + \sum_j p_j v_j, \quad o = \frac{u}{l}$$

Initialize m to negative infinity, ℓ to zero, and u to zero. Handle the first non-empty tile explicitly. Perform the final division after all key blocks. This computes the same attention function in real arithmetic. Where practical, reuse the blocked arithmetic and vectorization approach to support a meaningful comparison.

All three families retain quadratic work in N. The two matrix products require approximately $4N^2d$ floating-point operations, excluding softmax and scaling. Streaming scratch storage per worker is O(BrBc + Brd + Bcd). Count any additional packing buffers explicitly. Report the 16Nd bytes for Q, K, V, and O separately. Each additional worker needs its own scratch storage. Lower storage use does not necessarily mean lower DRAM traffic.

Task 4 Parallel execution and attribution

Parallelize the blocked materialized and streaming versions over separate query-row blocks. Workers share Q/K/V but have their own output rows and scratch buffers. Start with static scheduling. Document thread affinity, false sharing, and load balance. Keep each row's online recurrence within one worker. Work partitioning in [5] provides context. Its GPU-specific mechanisms are outside this CPU scope.

Compare all three families with one thread. Then measure thread scaling for each optimized family. On representative shapes, disable compiler vectorization in a controlled ablation to separate its contribution from tiling. If fusion prevents separate stage timings, use the total time per call as the main measure.

Minimum completion: Deliver three correct algorithm families and verify vectorization in suitable arithmetic loops. Both optimized versions must support threading. Correctness, runtime, and storage results must be reproducible. No fixed speedup is required.

4 Experimental designs

Correctness contract

Use deterministic FP32 inputs sampled from [−1, 1] for the core study. Before timing, validate every reported shape and implementation against the independent FP64 reference. Test zeros, identical rows, signed random values, and N = 1. Include non-divisible dimensions such as (N, d) = (127, 33) and (513, 65). Also test finite inputs with Q and K scaled by 10 to produce concentrated softmax distributions.

The initial acceptance rule is $|O - O_{ref}| \leq 10^{-4} + 10^{-3}|O_{ref}|$ for every output element. Report maximum absolute error and relative Frobenius error. Protect the denominator when the reference norm is near zero. Reject NaN/Inf outputs. Investigate failures and justify any tolerance change explicitly. Check different thread counts and tile boundaries. Use memory and race diagnostics where supported. Do not use sanitized builds for performance measurements.

Bounded experiment grid

Dimension	Planned cases
Sequence length N	128, 512, 2048, 4096
Feature dimension d	32, 64, 128; shared by Q, K, and V
Algorithm families	Materialized baseline, blocked materialized, streaming; all at one thread
Tile candidates	Br and Bc in {16, 32, 64}; hold feature blocking fixed and documented
Tuning subset	(N, d) = (512, 64) and (2048, 64); freeze one tile configuration per optimized family
Thread scaling	1, 2, 4, … up to physical cores, including the maximum; N = 512 and 4096, d = 64
Reproducibility	Fixed seeds; 5 warm-up calls and 15 timed repeats per configuration

The main grid has 12 shapes per family. Limit tuning to nine tile pairs per optimized family on the stated subset. Report the tuning cost and identify the tuning shapes. Use the selected tiles for thread-scaling runs. An N = 8192 memory-stress case is optional if resources permit. Set any resource cap before the main run and apply it consistently.

Timing and memory accounting

Preallocate buffers. Exclude input generation, allocation, validation, and file I/O from timing. Include all per-call packing, transposition, buffer initialization, softmax, output normalization, and threading overhead. Restore modified buffers between repeats. Time short calls in batches and retain observable outputs. Report the median and interquartile range. Interleave the variants to reduce bias from changes in system conditions over time.

Measure latency per attention call and speedup over the baseline for the same shape. Measure thread speedup against the same algorithm with one thread. Report peak live scratch bytes, persistent auxiliary storage, and input/output storage separately. Use process peak memory as a supplementary measure. Record the CPU model, cache sizes, core topology, ISA, compiler flags, affinity, and frequency policy. Describe cache conditions for repeated calls and keep them consistent.

Analysis and expected evidence

Plot runtime and scratch memory against N. Compare tile sizes and thread scaling. Report numerical errors and a representative optimization ablation. Profile one small and one large shape. For materialized versions, measure QK^T, softmax, and multiplication by V separately. Use profiler evidence to analyze fused streaming execution. Explain any slowdown from repeated K/V traversal, rescaling, small tiles, or thread overhead.

Use working-set estimates and available cache and cycle counters to assess memory and compute bottlenecks. Roofline [7] provides an analysis framework. A calibrated Roofline plot is optional. Keep allocated storage separate from measured memory traffic. The outcome will be recommendations supported by evidence across workloads. The project does not claim a new attention algorithm or guarantee acceleration.

5 Six-week work plan

Week	Work focus	Checkpoint
1	Freeze API, semantics, machine and inputs; implement baseline and FP64 reference.	Correct baseline and initial timings.
2	Develop blocked kernels and a small streaming prototype; verify recurrence.	Each route passes small and tail cases.
3	Integrate streaming and vectorization; run bounded tile tuning.	Three working families; vectorization evidence.
4	Add threads to both optimized routes; check ownership and profile.	Correct scaling runs and preliminary attribution.
5	Freeze configurations; run core experiments and analyze failures.	Raw data, plots and bottleneck explanations.
6	Reproduce from a clean build; complete report and demonstration.	Reproducible package and reviewed contributions.

Weeks are relative implementation weeks. The plan follows the six-week sample structure and will be aligned with the confirmed course schedule.

Proposed four member responsibilities

Member	Primary responsibility	Cross review
A	Attention semantics, baseline and independent reference	Online-softmax correctness
B	Blocked matrix kernels, packing and vectorisation	Timing boundaries and fairness
C	Streaming attention and memory accounting	Blocked-kernel indexing and tails
D	Threading, benchmark automation and profiling	Reference tests and reproducibility

Assign names after team agreement. All members contribute code, tests, and interpretation. Agree on a common kernel interface in week 1. Integrate work weekly and cross-review each implementation before benchmarking.

6 Optional extensions

Choose either one or both extensions after the core implementations and experiments are complete.

- **Causal masking:** Add causal masking to the materialized baseline and streaming version. Each query may attend only to its own position and earlier positions. Validate both versions against a masked FP64 reference. Compare runtime and scratch memory, and report causal results separately from unmasked results.
- **Optimized library comparison:** Add a CPU library reference that computes the same attention function. Match input shapes, precision, and thread counts. Include all required data conversions in the timing. Compare runtime and explain the main performance differences.

The core tasks remain the priority. Optional work will begin only after correctness and reproducibility are established.

7 Submission package

Deliver C++17 source, build instructions, automated tests, and a benchmark harness. Include raw CSV files, plotting scripts, and fixed configurations. Provide a small reproduction run that uses one command. The final report will cover design, correctness, experimental method, results, limitations, and individual contributions. The demonstration will include a correctness check and a representative comparison.

8 References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. “Attention Is All You Need.” Advances in Neural Information Processing Systems 30, 2017.

[2] M. Milakov and N. Gimelshein. “Online normalizer calculation for softmax.” arXiv:1805.02867, 2018.

[3] M. N. Rabe and C. Staats. “Self-attention Does Not Need O(n²) Memory.” arXiv:2112.05682, 2021; revised 2022.

[4] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.” Advances in Neural Information Processing Systems 35, 2022.

[5] T. Dao. “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning.” arXiv:2307.08691, 2023.

[6] K. Goto and R. A. van de Geijn. “Anatomy of High-Performance Matrix Multiplication.” ACM Transactions on Mathematical Software, 34(3), Article 12, 2008. doi:10.1145/1356052.1356053.

[7] S. W. Williams, A. Waterman, and D. A. Patterson. “Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures.” Technical Report UCB/EECS-2008-134, University of California, Berkeley, 2008.